

(Object Detection for Rescue Operations)

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of Winter
Training Program on Space Science Technology**

(Subject Name: Advanced drone Technology)

By

Sandeep Dhangar

Bharat Ratna Sardar Vallabh Bhai Patel, Rajkiya Engineering
College, Basti

Institute Roll no. = 2412130300047

Enrollment no. = 234215

Under the Supervision of
(Mr. Sandeep Kumar Khare)



**India Space Academy,
Department of Space Education, India Space Week**

Project Title

Object Detection for Rescue Operations

1. Introduction

- This project implements a real-time object detection system for a drone using YOLOv8 and OpenCV.
- The objective is to simulate a rescue drone that detects objects such as people or vehicles and applies a Target Lock when the object appears in the center region of the screen.

2. Tool & Technology Used

- Python
- OpenCV
- NumPy
- Ultralytics YOLOv8
- Torch

3. Working Logic

- The drone camera feed is captured using OpenCV.
- Each frame is passed to the YOLOv8 neural network model for object detection.
- Bounding boxes and confidence scores are extracted.
- Detections below 0.6 confidence are filtered out.
- The center of the bounding box is calculated using mathematical logic.
- If the object lies within the middle 20% of the screen width, the system activates Target Lock (Red Box). Otherwise, it remains in the Scanning Mode (Green Box).
- A simulated distance is calculated using bounding box height.
- FPS is displayed on screen as part of Drone HUD.

4. Confidence vs IOU

- Confidence represents the probability of correct detection.
IOU (Intersection over Union) measures overlap between bounding boxes and helps remove duplicate detections using Non-Max Suppression.

5. Code & Output

1. `import cv2`
2. `import numpy as np`
3. `import math`

```
4. import time
5. from ultralytics import YOLO
6.
7. # ----- Initialization ----- #
8.
9. model = YOLO("yolov8n.pt") # nano version for speed
10. CONF_THRESHOLD = 0.6
11.
12. def process_drone_feed(source=0):
13.     cap = cv2.VideoCapture(source)
14.
15.     prev_time = 0
16.
17.     while True:
18.         ret, frame = cap.read()
19.         if not ret:
20.             break
21.
22.         # Resize for better FPS
23.         frame = cv2.resize(frame, (640, 640))
24.
25.         results = model(frame, stream=True)
26.
27.         for r in results:
28.             boxes = r.boxes
29.
30.             if boxes is not None:
31.                 for box in boxes:
32.                     conf = float(box.conf[0])
33.
34.                     # Confidence Filtering
35.                     if conf > CONF_THRESHOLD:
36.
37.                         x1, y1, x2, y2 = map(int, box.xyxy[0])
38.                         cls = int(box.cls[0])
39.                         label = model.names[cls]
40.
41.                     # Center calculation using NumPy
42.                     center_x = (x1 + x2) / 2
43.                     screenwidth = frame.shape[1]
44.                     # Middle 20% logic
```

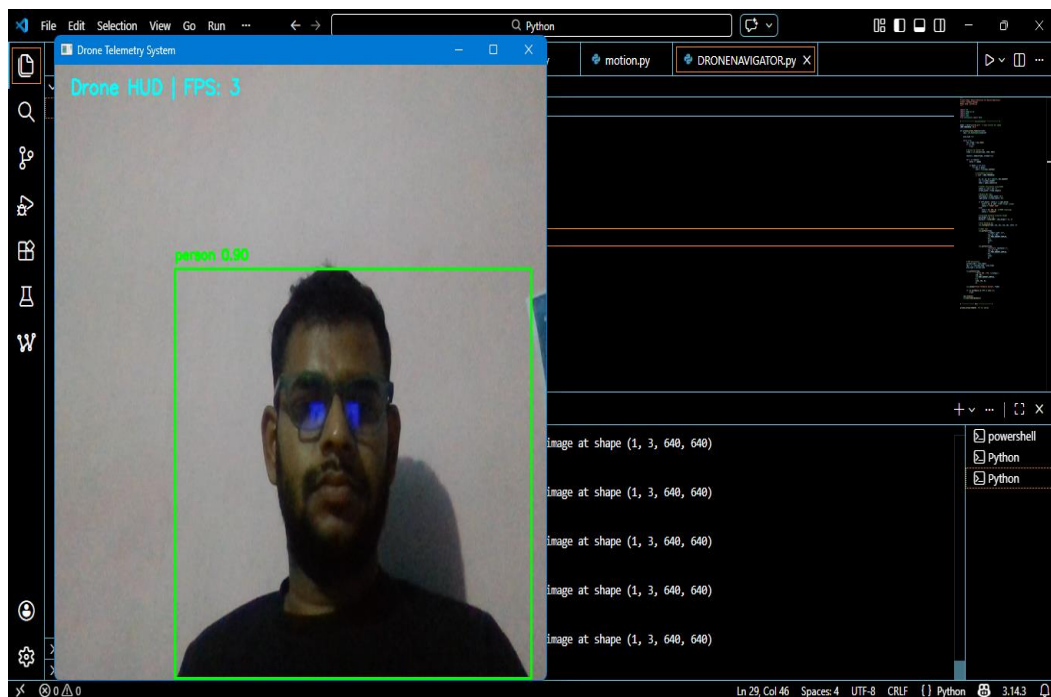
```
45. left_bound = screen_width * 0.4
46. right_bound = screen_width * 0.6
47.
48. if left_bound < center_x < right_bound:
49. color = (0, 0, 255) # RED (Target Locked)
50. status = "TARGET LOCK"
51. else:
52. color = (0, 255, 0) # GREEN (Scanning)
53. status = "SCANNING"
54.
55. # Simulated distance using box height
56. box_height = y2 - y1
57. distance = round(1000 / (box_height + 1), 2)
58.
59. # Draw bounding box
60. cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
61.
62. # Label text
63. cv2.putText(frame,
64. f"{label} {conf:.2f}",
65. (x1, y1 - 10),
66. cv2.FONT_HERSHEY_SIMPLEX,
67. 0.5,
68. color,
69. 2)
70.
71. cv2.putText(frame,
72. f"Distance: {distance} m",
73. (x1, y2 + 20),
74. cv2.FONT_HERSHEY_SIMPLEX,
75. 0.5,
76. color,
77. 2)
78.
79. # FPS Calculation
80. current_time = time.time()
81. fps = 1 / (current_time - prev_time)
82. prev_time = current_time
83.
84. cv2.putText(frame,
85. f"Drone HUD | FPS: {int(fps)}",
```

```

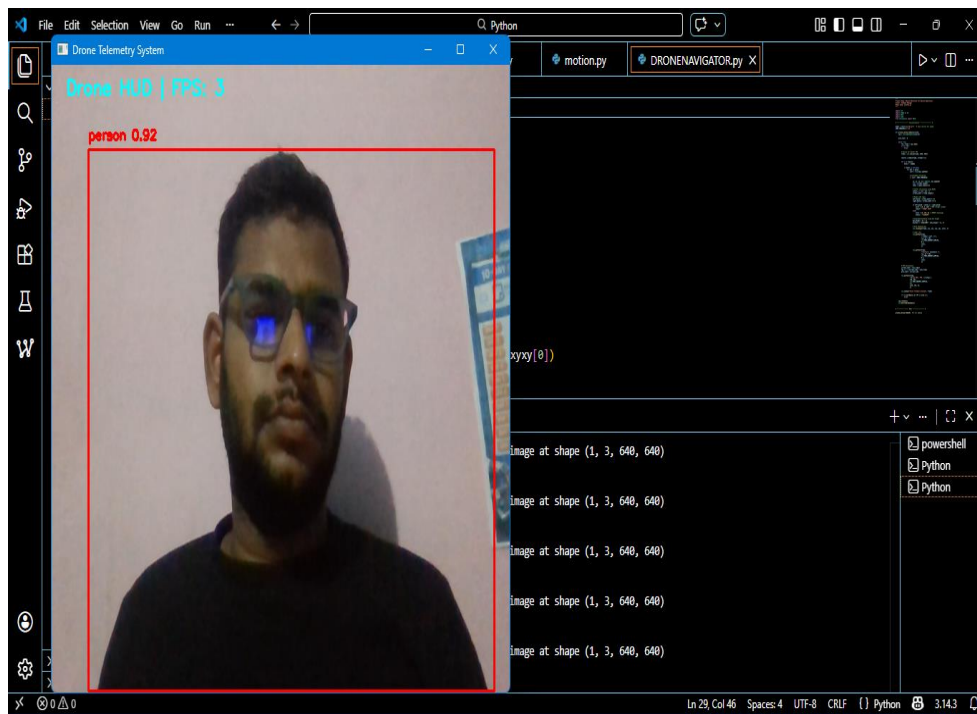
86. (20, 30),
87. cv2.FONT_HERSHEY_SIMPLEX,
88. 0.7,
89. (255, 255, 0),
90. 2)
91.
92. cv2.imshow("Drone Telemetry System", frame)
93.
94. if cv2.waitKey(1) & 0xFF == ord('q'):
95.     break
96.
97. cap.release()
98. cv2.destroyAllWindows()
99.
100.
101. # ----- Main ----- #
102.
103. process_drone_feed(0) # 0 for webcam

```

- **Scanning Mode (Green Bounding Box)**



- **Target Lock Mode (Red Bounding Box with Drone HUD)**



6. Conclusion

- The project successfully demonstrates real-time object detection for drone-based rescue operations. The integration of Deep Learning and Computer Vision enables intelligent decision-making and target tracking.